Search models, datasets, users...

← Back to Articles

A Guide to Reinforcement Learning Post-Training for LLMs: PPO, DPO, GRPO, and Beyond

Community Article Published January 19, 2026

▲ Upvote 13 +7

Karina Zadorozhny
karina-zadorozhny Follow

Definitions

Let's define standard reinforcement learning terms with an LLM setup in mind.

- **State** s_t: The current context which is the original user prompt and all tokens generated so far

  Example: Prompt: "The sky is..." → State: `["The", "sky", "is"]` in the token-space

- **Action** a_t: The next token being generated

  Example: `"blue"`

- **Policy** π_θ: The LLM itself. Which is essentially a probability distribution over all words in a vocabulary given a state s_t.

  Example: $\pi_\theta($ `"blue"` | `["The", "sky", "is"]` $) = 0.87$

- **Trajectory** τ: The full conversation. The complete sequence of states (context) and actions (tokens chosen) from the prompt to the end-of-sequence token.

- **Reward R**: The score. Usually assigned to the entire trajectory, indicating how good the full response was.
- **Critic Network $V(s)$**: A separate model (or head) that estimates the value of a state. It predicts how much future reward we're expecting to get given the current state. Also referred to as the Value Function.
- **Reward Model:** A separate model trained to learn human preferences or other rewards. Most commonly, it takes the full response and outputs the scalar reward R.
- **Reference model π_{ref}:** The starting, pre-trained LLM after SFT before any RL was used.

On-Policy Algorithms

In on-policy learning, the model actively generates its own data during training. The model generates responses. The responses get scored and the model's parameters updated.

The Core Objective

The core objective is to maximize the Expected Return $J(\pi_\theta)$.

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

This means we are computing the expected value of the reward across all conversations sampled from our model π_θ.

> *"Note that, strictly speaking, the policy π_θ outputs probabilities for a single token at a time. We write that we're sampling the full trajectory (sequence) from π_θ but in fact, we need some decoding strategy (like ancestral sampling or top-p sampling). So this is just a short-hand for saying that we are autoregressively generating the sequence step-by-step based on the policy."*

We want to find weights θ of our model that maximize the expected return J. To do this, we need to compute its gradients $\nabla_\theta J(\pi_\theta)$.

$$\nabla_\theta J(\pi_\theta) = \nabla_\theta \int P(\tau|\theta) R(\tau)\, d\tau$$

The problem is that can't differentiate through a discrete generation process, such as sampling tokens in our case. In a normal neural network, we can compute gradients for every step. But when an LLM generates text (or more generally, when an action is picked), it perfoms a sampling step that is non-differentiable. Moreover, the reward is often non-differentiable too (unless we use a differentiable reward model). To solve this, we use the score function estimator, also known as the log-derivative trick.

We first move the gradient inside (assuming regularity):

$$= \int \nabla_\theta P(\tau|\theta) R(\tau)\, d\tau$$

and apply the log-derivative trick.

$$= \int \underbrace{P(\tau|\theta) \nabla_\theta \log P(\tau|\theta)}_{\text{Replaced } \nabla_\theta P} R(\tau)\, d\tau$$

This let's us rewrite the gradient as an expectation and we can use sampling to estimate it.

$$= \mathbb{E}_{\tau \sim \pi_\theta} \left[\nabla_\theta \log P(\tau|\theta) \cdot R(\tau) \right]$$

This is great because we no longer need the derivative of the reward function and can estimate the gradient by sampling trajectories.

Now, for LLMs, the probability of the whole text is the product of probabilities for each token:

$$P(\tau|\theta) = \prod_{t=0}^{T} \pi_\theta(a_t|s_t)$$

We can take the log of the product, which becomes a sum

$$\log P(\tau|\theta) = \sum_{t=0}^{T} \log \pi_\theta(a_t|s_t)$$

and substitute it back to the equation above. We can also generalize the reward term $R(\tau)$ into a weight Φ_t:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \underbrace{\nabla_\theta \log \pi_\theta(a_t|s_t)}_{\text{Direction}} \cdot \underbrace{\Phi_t}_{\text{Weight}}\right]$$

where

- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ is the gradient of the log-probability. It tells us exactly how to update model's parameters to make this specific token a_t more or less likely (based on R) given its context s_t
- Φ_t is the weight of this update.

You can see different policy gradient algorithms as different implementations of the weight. On a high-level, you can see it as:

- If $\Phi_t = R(\tau)$, which is the total reward of the trajectory, we get REINFORCE (or more commonly, we compute rewards to-go $\Phi_t = \sum_{k=t}^{T} r_k$)
- If $\Phi_t = Q(s_t, a_t) - V(s_t)$, the Advantage, we get Vanilla Policy Gradient or Actor-Critic methods.

REINFORCE

REINFORCE is essentially a direct implementation of the equation above. In the simplest case, the weight of updates is set as trajectory's total reward \R(\tau)\. It is intuitive: if the model writes a response and gets a high score, we reinforce every token it used. If it gets a low score, we discourage them.

The update rule looks like this, averaged over a batch of trajectories.

$$\nabla_\theta J(\pi_\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot R(\tau)$$

where

- $\nabla_\theta \log \pi_\theta(a_t|s_t)$ is the gradient of the log-probability as above
- $R(\tau)$ is the score of the full response. It acts as a weight for the update. If the reward is positive, we move the weights with the gradient and increase the probability of the token. If the reward is negative, we suppress the probability of these tokens.

REINFORCE can be very inefficient. Imagine that a model generates a very good long response but hallucinates at the end, and the Reward Model assigns it a score of -100. REINFORCE will punish the entire generated text because it can't tell which token caused the score to be low. The biggest difficulty of this algorithm is high variance and instability.

PPO

Proximal policy optimization (PPO) is a family of policy gradient methods introduced by OpenAI in 2017 (Schulman et al. 2017), and fixes many of these instabilities. PPO is one of the most widely used algorithms in RL applications. It solves issues of Vanilla Policy Gradients and REINFORCE by using three main improvements:

- **Generalized Advantage Estimation (GAE):** PPO uses GAE for computing advantage values, which helps reduce variance in policy gradient estimates while maintaining low bias.
- **Actor-Critic interplay:** PPO introduces a critic model (also called the value function).
- **Clipped Updates:** PPO limits policy updates by using a clipped surrogate objective or by using an adaptive KL divergence penalty.

Advantage Estimation and Critic Network

Instead of using raw reward R, PPO uses **Advantage** $\hat{A}_t$. Advantage tells us how much better this specific action was compared to the expected baseline result.

Advantage also only looks at rewards obtained from step t and later. We ignore what happened before action a_t because the new token cannot influence the past.

Advantage is defined as

$$\hat{A}_t = Q(s_t, a_t) - V(s_t)$$

where

- $Q(s_t, a_t)$ are rewards-to-go. This is the actual total reward we got after taking action a_t.
- $V(s_t)$ is the average reward we usually get from this state. It is predicted by a separate network called the **Critic**.

This significantly reduces noise because we are normalizing the signal against a baseline and only consider future rewards.

More specifically, PPO uses **Generalized Advantage Estimation (GAE)** formulation from Schulman et al. (2015), which uses smooth, exponentially-weighted advantage estimates from many steps. This helps to reduce variance in policy gradient updates and stabilizes training. You can read a nice explainer on GAE here.

Another important aspect of the PPO loss is that it avoids updates that are too large. When we update the weights, we don't want to make too large changes at once, which would cause "policy collapse". The original PPO paper introduced two ways to do that: PPO-CLIP and PPO-KLPEN.

PPO-CLIP

PPO-CLIP aims to stay within the trust region of the model by clipping the updates at each step. If our current policy is π_θ and π_{old} is the policy from the previous time

stamp, we define the probability ratio $r_t(\theta)$ as:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{old}(a_t|s_t)}$$

The ratio measures how different the new model's probability is compared to the old one.

We then restrict the updates to not be too large:

$$L^{PPO\ CLIP}(\theta) = \mathbb{E}\left[\min(\underbrace{r_t(\theta)\hat{A}_t}_{\text{Unclipped}}, \underbrace{\text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t}_{\text{Clipped}})\right]$$

Where:

- $\hat{A}_t$ is the Advantage
- ϵ is a small hyperparameter, usually 0.2. It defines by how much we allow the model to update. For example, it can be 20% relative to the old model.
- $\text{clip}(\dots)$ would then force the ratio to be between 0.8 and 1.2.

PPO-KLPEN

The PPO paper also defines an alternative definition to the clipped surrogate objective that adds an explicit KL divergence penalty to the objective with an adaptive penalty coefficient.

$$L^{PPO\ KL}(\theta) = \mathbb{E}_t\left[r_t(\theta)\hat{A}_t - \beta \cdot \mathrm{D}_{KL}[\pi_{old}(\cdot|s_t), \pi_\theta(\cdot|s_t)]\right]$$

where $\mathrm{D}_{KL}[\pi_{old}(\cdot|s_t), \pi_\theta(\cdot|s_t)]$ is a forward KL divergence penalty defined as follows (see the section below for a full explainer on this KL penalty):

$$D_{KL}[\pi_{old}(\cdot|s_t)||\pi_\theta(\cdot|s_t)] = \sum_a \pi_{old}(a|s_t)\log\left(\frac{\pi_{old}(a|s_t)}{\pi_\theta(a|s_t)}\right)$$

The coefficient β is updated during policy updates. While the initial value is a hyperparameter, its starting value doesn't seem to be important since the algorithm

quickly adjusts it.

Compared to PPO-CLIP, this method imposes a softer constraint on the policy divergence. It can be more complex for implementation compared to clipping, and PPO-CLIP is more widely adopted in practice.

GRPO

PPO requires you to load several huge models into memory. The policy (the LLM we're training), the reference model (frozen pre-trained version of the model), a separate large Critic network, and often a reward model. This is really memory-intensive.

Instead, DeepSeek-R1 and V3 models skip the Critic network for calculating the baseline values. Instead of calculating the Advantage by subtracting the value $V(s_t)$ predicted by the Critic, GRPO samples responses for a given prompt and uses the mean score of the group as the baseline.

That is, given a group of sample responses, $\{o_1, o_2, ..., o_G\}$ (usually $G = 64$) from the current model, we get their scores $\{R_1, R_2, ..., R_G\}$, and compute the advantage for output i as:

$$A_i = \frac{R_i - \text{mean}(\{R_1, R_2, ..., R_G\})}{\text{std}(\{R_1, R_2, ..., R_G\})}$$

The final loss then becomes:

$$\mathcal{L}_{GRPO}(\theta) = -\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(r_i(\theta)A_i, \text{clip}(r_i(\theta), 1-\epsilon, 1+\epsilon)A_i\right) - \beta \cdot D_K\right.$$

Now, notice the KL divergence term in this equation. It is **different** (!) from the penalty used in the PPO definition. This penalty is between **the frozen reference model** π_{ref} and **the current model** π_θ**.** This distinction deserves a separate section on its own, as it is very often a confusing point even in many popular open source implementations of RL algorithms.

KL Divergence Clarification

There are two KL penalties you'll see around: Trust-Region KL and Drift KL. Their purpose is very different, but they often get mixed up. They also use the opposite KL types - one is forward and the other reverse.

Quick refresher on KL divergence types

KL divergence is asymmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$.

Forward KL is defined as

$$D_{KL}(P||Q) = \sum_{x} P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

It is mean-seeking in the sense that when we minimize Forward KL with respect to Q, we force Q to cover as much of P as possible. It covers all modes of P.

Reverse KL is defined as:

$$D_{KL}(Q||P) = \sum_{x} Q(x) \log \left(\frac{Q(x)}{P(x)} \right)$$

Reverse KL is mode-seeking. It tries to capture one mode well and ignores other modes of P. It is fine with having some areas completely uncovered.

Now, in RL for LLMs, you will see two types of KL penalties.

Trust Region KL Penalty

The purpose of this penalty is to **stabilize training** and make sure that updates at each step are not too different from the weights at the immediate previous step.

$$D_{KL}(\pi_{\theta_{old}}||\pi_{\theta}) = \mathbb{E}_{\mathbf{x}\sim\pi_{\theta_{old}}} \left[\log \frac{\pi_{\theta_{old}}(\mathbf{x})}{\pi_{\theta}(\mathbf{x})} \right]$$

This penalty is defined as **Forward** $KL(P||Q)$ where P is fixed, and we optimize the second argument π_θ. Because Forward KL is mean-seeking, we make sure that

the new policy does not assign low probability to things that the old policy thought were good. We essentially want to cover all valid regions of the old policy.

The name Trust Region comes from an earlier paper from Schulman et al., 2015 that introduced Trust Region Policy Optimization (TRPO).

This penalty is **subtracted** directly **from the loss** in the PPO-KLPEN definition.

Drift KL Penalty

Drift KL penalty is introduced as a regularizer that **prevents reward hacking**. It ensures that the model π_θ doesn't drift away too far from the original SFT-pretrained reference model π_{ref}.

The Reward Model is not perfect, and optimizing without any constraints leads to the model finding ways to hack it. In the extreme, the model can start to output gibberish that gets very high scores. To avoid that, we use the Drift KL penalty.

This penalty is **Reverse KL**:

$$D_{KL}(\pi_\theta||\pi_{\text{ref}}) = \mathbb{E}_{\mathbf{x}\sim\pi_\theta}\left[\log \frac{\pi_\theta(\mathbf{x})}{\pi_{\text{ref}}(\mathbf{x})}\right]$$

Why reverse KL? You can think of it as forcing the model to only respond with outputs that have good support in the original reference model (i.e keep using natural language and not gibberish). If π_θ generates something where $\pi_{\text{ref}} \approx 0$, this will lead to a massive penalty.

At the same time, we don't mind the model forgetting other modes of the original model like toxic responses from the internet.

Typically, this penalty is **subtracted** directly **from the reward function.**

$$R_{total}(s,a) = R_{model}(s,a) - \beta \log\left(\frac{\pi_\theta(a|s)}{\pi_{ref}(a|s)}\right)$$

This is how Standard RLHF or PPO-CLIP and PPO-KLPEN algorithms do it. Note that this Drift KL penalty is usually the one people talk about and is much more often

used.

Now, going back to the GRPO loss, you might notice that the KL term is inside the loss function. GRPO is an outlier here, and its formulation of the loss with group averages puts the KL regularization term directly in the policy update rule.

KL Estimators and Gradient Pitfall

In practice, we don't compute the KL penalties directly as that is infeasible. Instead, we use sample-based KL estimators. This estimation is a very important detail which has recently been scrutinized as different estimators lead to different issues.

Recent work, *A Comedy of Estimators* (Shah et al., 2026) and *On a few pitfalls in KL divergence gradient estimation* (Tang & Munos, 2025), has shown how the most popular methods, including those used in popular open-sourced libraries, often calculate the wrong gradients.

But first, let's look at 3 popular ways to estimate KL divergence:

1. **K1 Estimator** (Naive) is a Monte Carlo estimator derived straight from the definition. This estimator is unbiased and most honest but in practice has quite high variance.

$$\mathbb{KL}_1 = \sum_{t=1}^{T} \log \frac{\pi}{\pi_{\text{ref}}}$$

2. **K2 Estimator** is based on Taylor expansion of the KL divergence. Squaring the log ratio makes sure that penalty is always non-negative. Note that this makes it symmetric whereas KL is assymetric.

$$\mathbb{KL}_2 = \sum_{t=1}^{T} \frac{1}{2} \left(\log \frac{\pi}{\pi_{\text{ref}}} \right)^2$$

3. **K3 Estimator** was used in the PPO paper and is often the default estimator in libraries like TRL. Small deviations have almost no penalty. It is an unbiased estimator.

$$\mathbb{KL}_3 = \sum_{t=1}^{T} \left(\frac{\pi}{\pi_{\text{ref}}} - 1 - \log \frac{\pi}{\pi_{\text{ref}}} \right)$$

Now, as mentioned above, the Drift KL penalty can be either added to the reward (standard RLHF definition) or to the loss (GRPO and the likes).

- **In the Reward:** you calculate the KL value, then you **stop the gradients,** and then you subtract it from the reward
- **In the Loss:** when the KL penalty is in the loss function, it is passed to the differentiator and **included in the gradients.**

The pitfall is that when you differentiate a KL estimate in the loss, it actually does not equal to the gradients of the KL divergence term. So even if an estimator is unbiased, its gradients can be biased and wrong.

Shah et al., 2026 shows that the different configurations of the estimator type and whether it is included in the reward or the loss leads to dramatically different behavior and performance:

Configuration	Gradient Bias	Behavior	Performance	Findings & Explanation
K1 in Reward	Unbiased	Stable	Best	This is the gold standard of subtracting MC estimates from the reward. It outperforms other approaches on many reasoning tasks.
K1 in Loss	Zero in expectation	Training Instabilities	Poor	When you differentiate this term, the expectation of the gradient becomes zero and the model only receives noise. This leads to training instability.
K3 in Reward	Biased	Training Collapse	Failure	Biased gradient term with K3 estimator leads to a full

Configuration	Gradient Bias	Behavior	Performance	Findings & Explanation
				or partial collapse for all tested coefficient values.
K3 Loss	Biased	Stable	Good	This is what GRPO does and it works pretty well but worse than K1 in Reward.

Off-Policy Learning

Now let's move on to the off-policy methods, which actually do not have an explicit KL divergence term but model this implicitly.

On-policy methods can be computationally expensive. The bottleneck is the generation step. We also need to have multiple large models loaded at the same time (Actor, Critic, Reward Model, Reference Model). Off-policy methods learn from existing data and omit the need for a separate Reward Model.

Direct Preference Optimization

DPO (Rafailov et al., 2023) doesn't use a separate Reward Model neural network. Instead, it extracts the optimal reward directly from the preference data. DPO has been used in Llama 3 (combined PPO with DPO) and in Qwen-Chat models. Let's derive DPO's objective.

Step 1: Define the RLHF objective

First, let's revisit the standard RLHF objective with the KL term added to the reward calculation:

$$\text{Reward} = r(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$$

Our goal is to find a policy π that maximizes the expected reward while staying close to a reference model π_{ref}:

$$\max_{\pi} \mathbb{E}_{x\sim\mathcal{D},y\sim\pi}\left[r(x,y) - \beta \log \frac{\pi(y|x)}{\pi_{ref}(y|x)}\right]$$

Recap:

- x is the input (prompt) sampled from the dataset $\mathcal{D}$
- y is the output generated by the current LLM (the policy π)
- $r(x,y)$ is the reward function
- β is a parameter controlling the KL-divergence penalty

Step 2: Rewrite in the closed-form solution

To arrive at a closed-form solution, we can focus on the optimal policy π^* for the objective above. This is:

$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{ref}(y|x)\exp\left(\frac{1}{\beta}r(x,y)\right)$

The perfect policy is essentially the reference policy π_{ref} scaled by the reward $\exp\left(\frac{1}{\beta}r(x,y)\right)$.

$Z(x)$ is a partition function (normalization constant) to make sure probabilities sum to 1. We can't compute this term.

Step 3: Invert the equation

Normally, we'd train a Reward Model to get an estimate of the reward $r(x,y)$ and then use something like PPO to find the optimal policy π^*.

Instead, here we can solve the equation for the reward. This is the main DPO's trick:

$r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x)$

Step 4: Cancel out terms

In RLHF, we want to maximize the difference between good responses (winners) and bad responses (losers): $r(x,y_w) - r(x,y_l)$.

which we can rewrite using the equation above as

$$r(x, y_w) - r(x, y_l) = \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} + \beta \log Z(x)\right) - \left(\beta \log \frac{\pi_\theta(y_l|}{\pi_{ref}(y_l}$$

The advantage of doing so is that the partition function now cancels out in both of these expressions.

This gives us the final DPO loss:

$$\mathcal{L}_{DPO} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right]$$

The intuition behind this loss is that we want to increase the likelihood ratio of the winning responses and decrease the ratio of the losing responses, all the while staying close to the reference model.

DPO is memory-efficient and stable. However, it is quite sensitive to the distribution of the dataset and can easily overfit if the data is noisy.

Notice that even though the explicit KL term is gone, we still have the hyperparameter β which is now used to scale the log-probability ratios inside the sigmoid function. And in principle, it serves the same role as in PPO - higher values of β force the model to stay closer to the reference model and increase the implicit penalty.

Iterative and Online DPO

Offline DPO is typically trained on a static preference dataset.

However, as the model's training progresses and the policy π_θ changes, the model's output distribution starts to drift away from the provided data. The training would benefit from new preference relations that better correspond to its current abilities.

Iterative DPO aims to solve this shifting data distribution problem. It refines the model in multiple stages of data generation.

In iterative DPO, we do the following:

1. Generate new preference data (x, y_1, y_2) with the current model π_{θ_t}
2. Use the Reward model to label which response is better and assemble a new dataset D_t
3. Run a DPO step on D_t and update model to $\pi_{\theta_{t+1}}$

Identity Preference Optimization (IPO)

The sigmoid loss that DPO uses tries to push the probability gap between winning and losing responses to infinity. But sometimes, the losing response is not terrible, just not as good. One algorithm called IPO, replaces the sigmoid with a mean-squared error objective.

$$\mathcal{L}_{IPO} = \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\left(\log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} - \frac{1}{2\beta}\right)^2\right]$$

Instead of maximizing the gap to infinity, you can select some margin $\frac{1}{2\beta}$. Once the model reaches the gap, it stops updating. This essentially can act like a regularizer which prevents the model from overfitting to noisy data.

Kahneman-Tversky Optimization (KTO)

One problem of RLFH and DPO is that they require paired data of winners and losers (y_w, y_l). Data collected in real-world scenarios are often not paired unless an A/B test was ran. We just have good responses or bad responses that are not paired.

KTO, introduced in Ethayarajh et al., 2025, defines a new loss inspired by the Prospect Theory. Prospect Theory was introduced by Daniel Kahneman and Amor Tversky and describes how humans perceive losses and wins in an asymmetric manner, caring more about losses than wins. The KTO paper tries to use this

intuition and define a new loss that maps our human psychological model into it without the need of explicit pairs.

The KTO loss maximizes the utility of good responses and minimizes the utility of bad responses independently:

$$L_{KTO}(\theta) = \mathbb{E}_{x,y\sim D}\left[\underbrace{\lambda_y}_{\substack{\text{Class Weight} \\ (\lambda_D \text{ or } \lambda_U)}} - \underbrace{v(x,y)}_{\substack{\text{Value of} \\ \text{Generation}}}\right]$$

where the value function $v(x,y)$ is defined as:

$$v(x,y) = \begin{cases} \underbrace{\lambda_D \sigma\left(\beta(r_\theta(x,y) - z_0)\right)}_{\text{Value of Desirable Output}} & \text{if } y \text{ is Desirable} \\ \underbrace{\lambda_U \sigma\left(\beta(z_0 - r_\theta(x,y))\right)}_{\text{Value of Undesirable Output}} & \text{if } y \text{ is Undesirable} \end{cases}$$

We want the reward r_θ to be higher than z_0 if it's desirable and lower if undesirable relative to a reference point z_0. λ_D and λ_U are hyperparameters for desirable and undesirable outputs.

The reference point is defined in the paper as:

$$z_0 = \underbrace{\text{KL}(\pi_\theta(y'|x)||\pi_{ref}(y'|x))}_{\text{Average Drift from Ref Model}}$$

Note that the authors used an estimator, as is standard for the KL penalty calculation, or even set the reference point to zero when KTO is preceded by SFT on the same subset of data.

Simple Preference Optimization (SimPO)

All of the algorithms above require us to load a Reference Model π_{ref} into memory so that we can calculate the KL-divergence ratio. But these models are huge and it doubles the GPU memory requirement.

SimPO argues we don't need the reference model. Instead it aligns the model using length-normalized log probability of the tokens (with some margin target γ.

$$\mathcal{L}_{SimPO} = -\mathbb{E}\left[\log \sigma\left(\frac{1}{|y_w|}\log \pi_\theta(y_w|x) - \frac{1}{|y_l|}\log \pi_\theta(y_l|x) - \gamma\right)\right]$$

This can make it more memory-friendly and faster.

Summary

This post walks you through some of the basics of reinforcement learning for LLM. The field is moving very fast, so stay tuned for more updates and more specialized posts. Specifically, things not covered here include verifiable rewards, process vs outcome rewards, and search and inference-time compute.

Special thanks to Sabrina Mielke for her helpful feedback and review of this post.

By Karina Zadorozhny

Community

Edit Preview

Start discussing this article

Tap or paste here to upload images

Comment

Sign up or log in to comment

System theme

Company

TOS

Privacy

About

Careers

Website

Models

Datasets

Spaces

Pricing

Docs